ChatGPT

エグゼクティブサマリ

Cline リポジトリの `src` 以下は、VSCode拡張機能およびCLIで動作するAIコーディングエージェントの中核を構成します。主な構成要素は、**拡張機能エントリポイント**（`extension.ts`）から始まり、**Webview/UI** → **コントローラ** → **タスク実行** の流れで処理が進みます [1] [2]。設定管理やストレージ、エラーハンドリングなど初期化処理は `common.ts` の `initialize()` 関数で行われ、非同期的にエンドポイント設定や状態管理がセットアップされます [3] [2]。ユーザ操作（サイドバーでの入力など）は Webview が受け取り、メッセージがコントローラに伝搬、コントローラが「Plan/Act」二段階ワークフローでタスクを処理します（計画立案→ユーザ承認→実行） [4] [5]。主要モジュール間には、拡張機能／Webview層、コントローラ・タスク層、ホスト連携層、各種サービス層、共有ライブラリ層が存在し、Mermaid図で整理できます。

表1に `src` 配下のファイル・ディレクトリ一覧と役割を示し、以降各項目で詳細を解説します。

ファイル/ディレクトリ	役割概要
`core/`	中核ロジック。**controller/**（Webviewメッセージ処理・タスク管理）、**webview/**（Webview UI ライフサイクル）、**task/**（API呼び出しやツール実行）、**storage/**（状態管理）、**hooks/**（フックスクリプト）など。拡張機能の主要機能を提供。
`dev/`	開発支援スクリプト群。**commands/**（開発用CLIコマンド）、**grit/**（内部グリッド定義ファイル）。開発プロセスやビルドに関連するツール。
`exports/`	Cline拡張のAPIエクスポート定義。`index.ts`で外部向けの機能を公開し、型定義`cline.d.ts`も含む。 [6] [7]
`hosts/`	ホスト環境との連携モジュール。**vscode/**（VSCode固有の処理）、**external/**（外部プロセス用のgRPC等）、**host-provider.ts**（ホスト抽象化）など、VSCodeやCLIなど各環境からの呼び出しを処理。
`integrations/`	機能拡張・ツール連携モジュール群。OpenAI/Codex、GitHub、エディタ操作、ターミナル操作、チェックポイント、診断、通知など、外部サービスやユーティリティ連携機能を提供。
`packages/`	外部依存モジュールやラッパー。現状 `execa.ts`（子プロセス実行ラッパー）を含む。
`samples/cli/`	CLI使用例。CLI環境での設定例やワークフローサンプル（GitHub連携など）。
`services/`	各種バックグラウンドサービス。**auth/**（認証）、**account/**（アカウント管理）、**error/**（エラートラッキング）、**telemetry/**（統計収集）、**search/**、**glob/**、**ripgrep/**（ファイル検索）、**tree-sitter/**（構文解析）、**logging/**、**feature-flags/**、**mcp/**（モデルコンテキストプロトコル）、**temp/**、**browser/**、**banner/** など、多岐にわたる補助機能を提供。
`shared/`	複数プラットフォーム共通の機能・型。**proto/**（VSCode-ホスト間メッセージ定義）、**storage/**（ファイルストレージ管理）、**providers/**、**services/**、**utils/**、**messages/**、**multi-root/** など、拡張機能とCLIで共有されるコード。

ファイル/ディレクトリ	役割概要
`standalone/`	CLI単独動作用コード。**cline-core.ts**（コア起動）、**hostbridge-client.ts**（ホストブリッジ）、**lock-manager.ts**（排他制御）、**protobus-service.ts**（内部メッセージング）、**vscode-context.ts**（VSCodeと同様のコンテキストを再現）など。
`test/`	テストコード。**core/controller/**、**services/**、**hooks/** などのユニット／E2Eテスト群、シナリオテスト、テスト用ユーティリティを含む。
`types/`	型定義ディレクトリ。現状は `picomatch.d.ts` のみで、外部型補助用。
`utils/`	ユーティリティ関数群。ファイル操作、パス操作、Git操作、環境変数、再試行処理、文字列操作、タイムゾーン変換、ワークツリー管理など。多数の汎用モジュールとそのテストがある。
`common.ts`	**プラットフォーム共通の初期化・終了処理**。ログチャネル設定、エンドポイント設定読み込み、StateManager初期化、エラー/テレメトリサービス初期化、Webviewプロバイダ生成、定期同期・クリーンアップ起動などを実装 [3] [8]。
`config.ts`	エンドポイント設定管理。`ClineEndpoint` クラスで自己ホスト（オンプレ）モードとクラウドモードの判定・初期化を行う。`endpoints.json` のロードや検証、テレメトリ無効化などを担当 [9] [10]（自己ホスト機能）。
`extension.ts`	**VSCode拡張機能エントリポイント**。`activate()` 関数で環境設定、ストレージマイグレーション、共通初期化 (`common.initialize()`)、VSCode専用コマンド/ビュー登録などを行う [11] [2]。`deactivate()` で後始末 (`common.tearDown()`)。
`registry.ts`	拡張機能およびホスト情報の定義。拡張子ID/バージョン/コマンド名など (`ExtensionRegistryInfo`)、ホスト環境情報 (`HostRegistryInfo`) を初期化・取得するコードを含む [12] [13]。

エントリポイントと起動

- **VSCode拡張機能**: エントリポイントは `src/extension.ts` 内の `activate(context: ExtensionContext)` 関数 [11]。ここで最初に `setupHostProvider(context)` により VSCode 環境用のホストプロバイダを登録し、次に既存のストレージ移行（レガシーデータのクリーンアップ・エクスポート）を行います [14] [15]。その後、`common.initialize(storageContext)` が呼ばれ、先述した共通初期化処理（ログ設定、エンドポイント初期化、StateManager初期化、エラー/テレメトリサービスなど）が非同期に実行されます [3] [2]。初期化が完了すると、**サイドバーWebview** が生成され (`VscodeWebviewProvider`)、Hooksキャッシュの初期化やコマンド登録を経てユーザインタラクションの準備が整います [16] [17]。

- **CLI（コマンドライン）**: `cli/src/index.ts` の `Command` によってコマンドが定義されている（`commander` 使用）[18] [19]。CLI実行時はまず各種ユーティリティ・ホストプロバイダ・サービス（`HostProvider`、`AuthHandler`、`Session`、`StateManager`、`ErrorService` 等）を初期化し、入力（コマンド引数や標準入力）に応じてタスク処理ループを開始します。内部では `StandaloneTerminalManager` や `CliWebviewProvider`（CLI上の擬似UI）、`CliCommentReviewController` などが利用され、VSCode拡張と同様に**Plan/Act**ワークフローでタスクを処理します [20] [21]。最後に `disposeCliContext()` で StateManager の同期保存やリソース解放を行い終了します [22]。

メイン処理の実行フロー

Cline の主要フローは、「入力（ユーザ操作やコマンド）」→「Webview/UIからメッセージ受信」→「コントローラ処理」→「タスク実行（AI/ツール呼び出し）」→「出力（UI更新/ファイル変更）」という形で進みます。

1. **ユーザ入力**: サイドバーやコマンド実行でユーザがリクエストを発生させる（例: サイドバーの質問入力、Gitコミットメッセージ生成コマンドなど）。
2. **Webview・UIレイヤ**: VSCode拡張では Webview (React/HTML UI) がユーザ入力を受け取り、`postMessage` 等で拡張機能ホスト側に送信します。CLIではコマンド引数や標準入力が直接パースされます。
3. **コントローラ層**: 拡張機能では `WebviewProvider` が受信メッセージを `MessageController`（あるいは特定のコントローラ）にルーティングし、各タスク処理をトリガします。コマンド発行フックもここで実行されます。**TaskController** がメインのタスク管理を担当し、要求に応じて複数ステップのワークフローを実行します。
4. **Plan/Act フロー**: TaskController は2段階プロセスで処理します。まず Plan フェーズでタスクを分析し、計画を立案します（例: 「変更内容からのコミットメッセージ生成草案」）[4]。計画はユーザに提示され、ユーザからの承認・フィードバックを待ちます[4]。承認が得られれば Act フェーズに入り、計画に従って実際の処理（ファイル編集、Git操作、外部API呼び出しなど）を行います[5]。計画が棄却された場合は中断・別アクションにフォールバックします。
5. **API・ツール呼び出し**: Actフェーズでは外部AIモデルへのリクエスト（OpenAI, Anthropic, 自社APIなど）やツール実行（ファイル書き換え、コマンド実行）を同期・非同期に行います。例: リポジトリファイルの解析には `RipgrepService`、コードの変更にはエディタープロバイダ、外部ブラウザ操作には `BrowserService`、ファイル追加には GitHub PR 統合等、各種サービスモジュールが呼び出されます。処理は `async/await` が多用され、非同期に実行されます。
6. **例外・エラー処理**: `common.initialize` では初期化失敗時にログ出力やユーザ通知を行い[23]、各サービスでもエラー監視（`ErrorService`）が動作します。タスク中にエラーが発生すると、`TaskController` は例外捕捉してユーザへエラーメッセージを返します。作業キャンセルやリトライなど、柔軟な分岐も可能です。非同期処理は並行・逐次動作を適切に制御し、必要に応じてロックや同期を用います（例: `LockManager`）。
7. **UI更新と出力**: 最終的に、処理結果（生成テキストやファイル差分など）を Webview に返し、サイドバーを更新します。場合によっては別ビュー（コミットメッセージ入力パネル、Diffビューなど）も自動で表示します。CLIでは標準出力やエディターファイルへの書き出しで結果を返します。

全体的に非同期処理が中心であり、データの流れはメッセージ駆動、Promise ベースの処理となっています。図示すると、Extension → Webview → Controller → Task → Services/Integrations の順で依存が流れ、必要に応じて Shared モジュールを参照します[1][24]。

```
graph LR
    Ext[Extension<br/>(extension.ts)] --> WV[Webview(UI)]
    WV --> Ctrl[Controller<br/>(TaskController など)]
    Ctrl --> Task[Task実行エンジン]
    Task --> Services[各種サービス<br/>(Auth, Search, Telemetry, etc.)]
    Task --> Int[Integrations<br/>(Git, Editor, Browser, etc.)]
    Ctrl --> Shared[Sharedライブラリ]
    Services --> Shared
    Int --> Shared
    Ext --> Hosts[Hosts プロバイダ<br/>(vscode/external)]
    Hosts --> Ctrl
```

主要モジュール間の依存関係図（Mermaid形式）

```
graph LR
    subgraph VSCode_Extension
        E[Extension: extension.ts]
        WV[Webview: core/webview]
        Ctr[Controller: core/controller]
        T[Task Engine: core/task]
    end
    subgraph CLI
        CLI[CLI Entry: cli/src/index.ts]
        CLI_WV[CLI View: cli/controllers/CliWebviewProvider]
        CLI_Ctr[CLI Controller: cli/controllers/...]
    end
    subgraph Host
        VSCodeProv[VSCode Host: hosts/vscode]
        ExtProv[External Host: hosts/external]
        HostProv[HostProvider (抽象)]
    end
    subgraph Services
        AuthSvc[Auth Service]
        ErrorSvc[Error Service]
        Telemetry[Telemetry Service]
        Search[Search Service]
        TempSvc[Temp/Cleanup Service]
        Hooks[HookDiscovery/Process]
    end
    subgraph Integrations
        GitInt[GitHub/Git Integration]
        EditInt[Editor/FileEdit]
        BrwInt[Browser Integration]
        TermInt[Terminal Integration]
    end
    subgraph Shared
        Storage[Storage (StateManager)]
        LoggerSvc[Logger]
        Proto[Proto Messages]
        Utils[Utils Libraries]
    end

    E --> WV
    WV --> Ctr
    Ctr --> T
    T --> AuthSvc
    T --> Search
    T --> GitInt
    T --> EditInt
    T --> BrwInt
    T --> TempSvc
    AuthSvc --> Shared
```

```
    ErrorSvc --> Shared
    Telemetry --> Shared
    Ctr --> Shared
    CLI --> CLI_Ctr
    CLI_Ctr --> CLI_WV
    CLI_WV --> T
    HostProv --> E
    HostProv --> CLI
    VSCodeProv --> HostProv
    ExtProv --> HostProv
    Hooks --> Shared
    Storage --> Shared
```

重要な関数・型と解説

- **`activate(context: vscode.ExtensionContext)`** (`src/extension.ts`) [11]: 拡張機能の初期処理開始ポイント。VSCode用ホストプロバイダ設定、既存データの移行・クリーンアップ、`common.initialize()` 呼び出しを含む。例:

```typescript
export async function activate(context: vscode.ExtensionContext) {
  // VSCodeホストプロバイダの設定
  setupHostProvider(context);
  // レガシーデータのクリーンアップ
  await cleanupLegacyVSCodeStorage(context);
  // ストレージのエクスポート
  const workspacePath = vscode.workspace.workspaceFolders?.[0]?.uri.fsPath;
  const storageContext = createStorageContext({ workspacePath });
  await exportVSCodeStorageToSharedFiles(context, storageContext);
  // 共通初期化（webview生成等）
  const webview = await initialize(storageContext) as VscodeWebviewProvider;
  // ...
}
```

  最後に各種コマンドやビューの登録を行う。

- **`initialize(storageContext: StorageContext): Promise<WebviewProvider>`** (`src/common.ts`) [3] ([25] raw): 拡張機能/CLI共通の初期化処理。設定ファイル読み込み（`ClineEndpoint.initialize`）や `StateManager` 初期化を行う。例:

```typescript
export async function initialize(storageContext: StorageContext):
Promise<WebviewProvider> {
  // 設定の読み込み
  const { ClineEndpoint } = await import("./config");
  await ClineEndpoint.initialize(HostProvider.get().extensionFsPath);
  // ストレージの初期化（失敗時はエラーメッセージ表示）
  try {
    await StateManager.initialize(storageContext);
  } catch (error) {
```

```
    Logger.error("[Cline] CRITICAL: Failed to initialize StateManager:", error);
    HostProvider.window.showMessage({ type: ShowMessageType.ERROR, message:
"Failed to initialize storage..." });
  }
  // ErrorService, Telemetry, Webview生成など
  await ErrorService.initialize();
  const webview = HostProvider.get().createWebviewProvider();
  // Background sync開始、古い一時ファイル削除など
  syncWorker().init(...);
  ClineTempManager.startPeriodicCleanup();
  FileContextTracker.cleanupOrphanedWarnings(StateManager.get());
  telemetryService.captureExtensionActivated();
  return webview;
}
```

これによりエンドポイント設定・共有ストレージが整備される [3] [24]。

- **`ClineEndpoint` クラス** (`src/config.ts`) [9] [10]: 環境設定のシングルトン。`initialize()` メソッドでバンドル済み/ユーザ設定の `endpoints.json` を読み込み、自己ホストモード判定を行う [9]。`isSelfHosted()` は初期化前も `true` を返すフェイルセーフ設計。自己ホスト時はテレメトリ無効化や自動更新停止などの措置を行う [26] [10]。

- **Plan/Act ワークフロー (`TaskController`)**: タスク実行は2段階に分けられる [4] [5]（Clineブログ記事より）。擬似コード例:

```
export class TaskController {
  async processTask(task: Task): Promise<Result> {
    // Planフェーズ：要求分析と計画立案
    const plan = await this.planPhase(task);
    await this.presentPlanToUser(plan);
    const userApproval = await this.getUserApproval(plan);
    if (!userApproval) {
      return this.handlePlanRejection(plan);
    }
    // Actフェーズ：計画に基づく実行
    const executionResult = await this.actPhase(plan);
    return executionResult;
  }
}
```

この設計により、ユーザがAIの判断に介入・制御しながら処理を進められる [4] [5]。

- **`tearDown()`** (`src/common.ts`) [27]: 拡張機能終了時のクリーンアップ処理。例:

```
export async function tearDown(): Promise<void> {
  // フックプロセスやwebviewインスタンス、テレメトリなどを解放
  await WebviewProvider.disposeAllInstances();
  syncWorker().dispose();
```

```
  clearOnboardingModelsCache();
  await HookProcessRegistry.terminateAll(); // 実行中フックスクリプト停止
  HookDiscoveryCache.getInstance().dispose(); // フック発見キャッシュ解放
  ClineTempManager.stopPeriodicCleanup();
  // ...
}
```

実行中フックの強制停止やキャッシュ破棄によりプロセス間の残留を防いでいる [28]。

- **その他重要型**: `ExtensionRegistryInfo`（拡張ID/コマンド一覧）、`HostRegistryInfo`（ホスト環境情報） [12] [13]。プログラム全体の設定値やコマンド定義を保持するための定数群。

入力→処理→出力のデータフロー例

- **例：Gitコミットメッセージ生成**
- **入力**: ユーザがGit変更を用意し、サイドバーで「Generate Commit Message」ボタンをクリック（またはCLIで `cline commit create` コマンド実行）する。
- **処理**: Webviewがユーザのクリックを検知しメッセージを拡張機能に送信。`TaskController` が変更内容を分析し、Planフェーズで提案メッセージを生成（例: GPTベースでDiffを要約）しサイドバーに表示 [4]。ユーザが承認すれば、Actフェーズで実際にコミットメッセージファイル（またはコミットコマンド）に追加する処理を行う。処理途中での非同期呼び出し（AI API、ファイルI/O）は `await` で扱われる。
- **出力**: 生成されたコミットメッセージがUIに返され、自動でコミットログ欄に挿入されるかCLIでは標準出力に表示される。変更はGitに反映される（コミット実行） [5]。

- **例：CLIでの対話型タスク**

- **入力**: 端末で `cline task run "リファクタリング"` （架空）と入力。
- **処理**: CLIはPlan/Act モードを選択し、最初に「変更案生成のみ」か「実行まで行うか」をユーザに確認する。承認するとCLI上で計画内容を段階的に表示しつつ、内部的には `TaskController` が生成した命令を順次実行する。必要に応じてVSCode外の編集（`FileEditProvider`）やシェルコマンドを起動する [20] [29]。処理結果を画面にログとして逐次出力する。
- **出力**: 最終的に処理済みファイル内容やコンソール出力、生成されたファイル・コミットなどがユーザに返される。自動更新や結果のサマリはテキスト出力される。

これらは1つの代表例ですが、基本的に「入力メッセージ→AI/ツール処理→出力メッセージ/操作」の流れを全てClineが仲介・記録します。

設計上の特徴・改善提案

- **透明なPlan/Act設計**: Clineはタスクを計画と実行に分けることで、ユーザがAIの意思決定過程を把握・介入できるようにしている [4] [5]。これにより「魔法のような」コード変更でなく、逐次確認可能なワークフローとなっている。
- **自己ホストモード対応**: `ClineEndpoint` を介して `endpoints.json` を読み込むことで、企業内環境向けのオフラインモードを提供 [26]。自社APIへの切り替えや分析サービス停止などを自動で行い、データ主権を尊重する設計は大規模導入に適する。
- **フック機構**: `core/hooks` 以下により、ユーザ指定のスクリプトをPlan/Actの各フェーズに挟める。`HookProcessRegistry` による実行管理と `HookDiscoveryCache` によるキャッシュで高速化している [28]。

- **マルチルート/ワークツリー対応**: VSCodeの複数フォルダやGit Worktree機能に対応し、必要に応じて自動的にサイドバーを開く処理もある [24] [30]。これにより複数リポジトリを跨いだ開発でも文脈維持が可能。
- **改善提案**: 現在のWebviewベースUIは強力だが、複数タブや大規模プロジェクトではインスタンスが増えメモリ負荷が課題になる可能性がある（Issue #3851 参照）。非同期/イベント駆動の性質上、競合状態やタイミングのずれにも注意が必要。エラーハンドリング強化（例: ネットワーク切断時のリトライ）、ログ監視、プラグインアーキテクチャによる拡張性向上がさらなる安定性向上に寄与するでしょう。ユーザからの期待値として、計画生成フェーズをスキップするオプション（フル自動実行モード）なども要望されており、将来的な機能追加の余地があります。今回の調査で得た情報は以上です。

参考資料: Clineリポジトリ内ソースコードおよび公式ドキュメント [1] [3] [9] [12] [28] より作成。脚注先のコード参照にて詳細を確認できます。

---

[1] cline/src/core at main · cline/cline · GitHub
https://github.com/cline/cline/tree/main/src/core

[2] [11] [14] [15] [16] [17] cline/src/extension.ts at main · cline/cline · GitHub
https://github.com/cline/cline/blob/main/src/extension.ts

[3] [8] [23] [27] [28] [30] raw.githubusercontent.com
https://raw.githubusercontent.com/cline/cline/refs/heads/main/src/common.ts

[4] [5] 颠覆式AI编程代理：Cline如何解决开发者三大核心痛点的创新实践 - AtomGit | GitCode博客
https://blog.gitcode.com/7b62ddc4bca5e033b74501e28b491aff.html

[6] [7] cline/src/exports at main · cline/cline · GitHub
https://github.com/cline/cline/tree/main/src/exports

[9] [10] [26] How to Deploy Cline in a Self-Hosted or On-Premise Environment | cline | Instagit
https://instagit.com/cline/cline/how-to-deploy-cline-in-self-hosted-or-on-premise-mode/

[12] [13] cline/src/registry.ts at main · cline/cline · GitHub
https://github.com/cline/cline/blob/main/src/registry.ts

[18] [19] [20] [21] [22] [29] cline/cli/src/index.ts at main · cline/cline · GitHub
https://github.com/cline/cline/blob/main/cli/src/index.ts

[24] [25] cline/src/common.ts at main · cline/cline · GitHub
https://github.com/cline/cline/blob/main/src/common.ts